GPU Computing
Exercise 1

Luis Wenzel
Miro Joensuu
Daniel Bayer
Noah Schneider

1 Reading

1.1 A bridging model for parallel computation.

The article describes the model of the bulk synchronous parallel computer (BPSC), which is meant to be a briding model decoupling the hardware and software aspects of parallel computation. The BPSC model is described as consisting of three components: a set of processing and memory units, a router to deliver messages between those units and a synchronization mechanism to coordinate sequences of computation steps (supersteps) globally. The focus of the article is put on dealing with the distribution of memory contents needed in the local computation steps, as well as the networking.
The article illustrates, how an efficient usage of the different components could be achieved with regards to parallelism, for example by proposing hashing functions to deal with questions of data distribution or starting a much higher number of processes then processors are available to create parallel slackness. It thus provides a hardware-agnostic model, which then would enable to reason about the efficiency of parallel algorithms without needing to take into account the specific details of the available architecture.
In our opinion, the BSPC model gives useful abstraction ideas, many of which are still relevant today. The simplicity of the undelying idea provides a good basis to reason about the basic challenges of parallel computation, e.g. data distribution and communication. However, it is limited in capturing the complexity of modern architectures, especially with regards to memory hierarchies and communication patterns.

1.2 Evolution of the Graphics Processing Unit (GPU).

The "Evolution of the Graphics Processing Unit" article describes different milestones of the GPU development with the example of NVIDIAs evolution from the GeForce 256 to the Ampere A100 GPU. There you could see how the domain specific 3d-graphic-processors became modern highly parallel arithmetic units for deep learning and high-performance computing.
The high performance at floating point calculations made the GPU the perfect candidate for scientific computing. This was further accelerated through CUDA programming, even higher parallelism and specialized hardware for matrix calculations and raytracing. GPU inference performance improved by more than double each year and therefore exceeds what Moore's law scaling would predict.
In our opinion the article presents a good overview of the technological improvement of NVIDIA GPUs and the versitile usage of these new powerful GPUs in for example high performance computing, deep learning or raytraced graphics. This article highlights the devolopment of NVIDA graphic cards. Since it is only about NVIDA GPUs, it would be helpful, if other companys GPUs would also be included in such review to get a deeper understanding of thel technological progress of the last years.

2 Amdahl's Law

2.1 Deriving Amdahl's Law

Amdahl's Law gives a speedup S for a parallel execution T_p with respect to a serial execution T_s as:

$$S = \frac{T_s}{T_p}$$

T_p consists of a parallel fraction p and a serial fraction $(1-p)$, so that for n processors:

$$T_p = (1-p)T_s + \frac{pT_s}{n}$$

Inserting this into the speedup equation gives:

$$S = \frac{T_s}{(1-p)T_s + \frac{pT_s}{n}} = \frac{1}{(1-p) + \frac{p}{n}}$$

2.2 Correctness

Since it assumes a perfectly even distribution of the problem without any overhead for distributing and collecting the work or communcation between

the processors, Amdahl's law will give an upper bound for most computations.
For example, when executing a sorting algorithm in parallel, all of the data needs to be distributed and collected, while not much computation is done per data element. In this case, the speedup will be much lower than predicted by Amdahl's law.
There are also cases where the speedup can be higher than predicted. An example would be a matrix multiplication, where only after the data has been distributed to the processors, the data then might fit fully into the caches, leading to a superlinear speedup.

2.3 Calculating speedup

2.3.1 a

$$S = \frac{1}{(1 - 0.5) + \frac{0.5}{4}} = 1.6$$

2.3.2 b

$$S = \frac{1}{(1 - 0.2) + \frac{0.2}{8}} = 1.\overline{21}$$

2.3.3 c

The lower speedup achieved in the second example shows, that a lower parallel fraction of the code has a big impact on the maximum achievable speedup, which cannot be compensated by just increasing the number of processors. This illustrates the need to focus on keeping the sequential part of the code as low as possible, rather than just scaling up the hardware.

3 Present